

成员 A：数据工程与数据库 — 报告章节

项目名称：Agentic BI 驱动的多表电商运营分析与决策智能系统

负责模块：数据集整理、数据清洗、MySQL 建库导入、预聚合层设计与性能对比实验

撰写人：陈子昂

1. 数据集说明

1.1 数据来源与规模

本项目采用 Kaggle 公开数据集 **Brazilian E-Commerce Public Dataset by Olist**（巴西 Olist 电商平台交易数据）。数据覆盖 **2016 年 9 月至 2018 年 10 月** 的真实订单链路，包含订单、明细、商品、客户、卖家、支付、评论、地理坐标及品类翻译等 **9 张独立业务表**，通过 `order_id`、`customer_id`、`product_id`、`seller_id` 等字段关联，适合开展跨表 JOIN、多维聚合与 Agent 自然语言查数实验。

经本组清洗后入库的数据规模如下：

表名	清洗后行数	主要用途
orders	99,441	订单主表、时间轴、配送时效
order_items	112,650	价格、运费、GMV 计算
customers	99,441	客户地域、去重用户数
products	32,951	品类、商品属性
sellers	3,095	卖家地域与绩效
payments	103,886	支付方式、分期
order_reviews	99,067	评分与评论文本（供 NLP Agent）
geolocation	720,154	邮编前缀与经纬度（地图可视化）

product_category_name_translation	71	葡语品类 → 英语品类
-----------------------------------	----	-------------

1.2 表间逻辑关系

核心关联路径简述如下：

- `orders` $\longleftrightarrow$ `order_items`：一对多（一单多行明细）
- `orders` $\longleftrightarrow$ `customers`：多对一（客户下单）
- `order_items` $\longleftrightarrow$ `products` / `sellers`：多对一
- `orders` $\longleftrightarrow$ `payments` / `order_reviews`：一对多
- `products` $\longleftrightarrow$ `product_category_name_translation`：品类名翻译
- `customers` / `sellers` $\longleftrightarrow$ `geolocation`：通过邮编前缀（zip code prefix）近似关联地理坐标

1.3 分析口径约定

为便于 Agent 与预聚合层一致，本组约定：

- **GMV（成交总额）**：在 `order_items` 层按 `SUM(price + freight_value)` 汇总；与课件及作业说明一致。
- **准时送达**：`order_delivered_customer_date <= order_estimated_delivery_date` 记为准时；仅统计同时具备下单、签收、预估送达时间戳的订单。
- **时间字段**：趋势分析默认以 `orders.order_purchase_timestamp` 为时间轴。

1.4 数据特点与注意事项

- 多表结构考察数据建模与 JOIN 能力；部分关联会产生较大中间结果集，**必须**配合预聚合层加速 Agent 查询。
- 品类名、评论文本为**葡萄牙语**；品类分析需 JOIN 翻译表得到 `product_category_english`。
- Kaggle 原版 `products` 表中字段名为 `product_name_lenght`、`product_description_lenght`（拼写 typo），库表与 CSV **保持一致**，避免导入失败。

2. 数据预处理步骤

预处理脚本：`utils/data_cleaning.py`。流程为：从 `data/raw/` 读取 Kaggle CSV → 按表清洗 → 输出 UTF-8 编码的 `data/clean/*.csv`，供 `utils/db_init.py` 导入 MySQL。

2.1 总体流程

```
1  Kaggle 原始 CSV (data/raw/)
2      ↓ data_cleaning.py
3  清洗后 CSV (data/clean/)
4      ↓ db_init.py
5  MySQL 基表 + mv_* 预聚合表
```

2.2 各表清洗规则

表	主要处理
orders	主键与状态字段去空格；5 个时间列转 datetime（非法 → NaT → 入库 NULL）；按 <code>order_id</code> 去重
order_items	数值列 <code>price</code> 、 <code>freight_value</code> 转数值、缺失填 0； <code>shipping_limit_date</code> 转时间；按 <code>(order_id, order_item_id)</code> 去重
customers / sellers	字符串 strip；按主键去重
products	品类空值保留 NULL；数值属性转 numeric；保留 Kaggle 原版 <code>*_lenght</code> 列名
payments	分期、序号、金额转数值；按 <code>(order_id, payment_sequential)</code> 去重
order_reviews	评分转整型；评论文本 strip；时间列转 datetime；按 <code>review_id</code> 或 <code>(order_id, review_creation_date)</code> 去重
geolocation	经纬度转数值，剔除无效坐标；按 <code>(zip_prefix, lat, lng)</code> 去重以降低冗余行数
product_category_name_translation	葡/英品类名 strip；按葡语品类名去重

2.3 缺失值与异常值策略

- 时间字段：无法解析的值转为 `NaT`，导入 MySQL 后为 `NULL`，不参与依赖该时间的聚合（如 `WHERE order_purchase_timestamp IS NOT NULL`）。
- 金额字段：`order_items` 中缺失价格/运费按 0 处理，避免 JOIN 后 SUM 出现 NaN。
- 地理坐标：经纬度缺失或非法的行直接删除。
- 重复行：各表按业务主键或复合键 `drop_duplicates(keep='first')`，保留首条记录。

2.4 复现命令

```
1 cd AgenticBI_Final_Olist
2 pip install -r requirements.txt
3 python utils/data_cleaning.py
4 python utils/db_init.py
```

3. 数据库表结构说明

3.1 环境与命名

- 数据库名：`olist_agentic_bi`（默认，可通过 `MYSQL_DATABASE` 覆盖）
- 字符集：`utf8mb4`（支持葡语评论及 emoji）
- 存储引擎：InnoDB
- DDL 脚本：`utils/schema_ddl.sql`

3.2 九张基表概要

表名	主键	核心字段	索引设计要点
orders	order_id	客户、状态、5 个时间戳	customer_id、status、purchase 时间
order_items	(order_id, order_item_id)	product_id, seller_id, price, freight_value	product_id, seller_id

customers	customer_id	unique_id, 城市, 州, 邮编前缀	customer_state
sellers	seller_id	城市, 州, 邮编前缀	seller_state
products	product_id	品类, 名称/描述长度, 重量尺寸	—
payments	(order_id, payment_sequential)	payment_type, installments, payment_value	payment_type
order_reviews	review_id (自增)	order_id, score, 标题/正文, 时间	order_id, review_score
geolocation	—	邮编前缀, lat/lng, 城市, 州	zip_prefix
product_category_name_translation	product_category_name	英语品类名	—

说明：DDL 中未强制声明外键约束（`FOREIGN_KEY_CHECKS=0` 后建表），便于批量导入；逻辑关联由应用层与 Agent 生成的 SQL 保证。

3.3 数据字典

面向 Agent / NL2SQL 的字段说明维护于 `config/data_dictionary.yaml`，内容包括：

- 各基表粒度、主键、关键列含义；
- GMV、准时率等业务口径；
- 六个 `mv_*` 预聚合表的粒度、字段及适用问题类型。

4. 预聚合视图设计

4.1 设计动机

Olist 九表关联查询在 Agent 高频调用场景下，若每次对 `orders` JOIN `order_items` 等基表做实时聚合，将产生数万行级中间结果，响应延迟明显。作业要求预先创建 **Pre-Aggregation 加速层**；本项

目在 MySQL 中以物理表 `mv_*` 实现（MySQL 无原生物化视图时的等价方案），由 `utils/pre_aggregation.sql` 一次性计算写入，可通过 `db_init.py` 或单独执行该脚本刷新。

4.2 六个预聚合表

视图名	粒度	核心指标	典型问题
mv_monthly_sales	年-月	total_gmv, total_orders, avg_basket, total_freight	月度 GMV 趋势、环比
mv_state_sales	年-月 × 客户州	total_gmv, total_orders, unique_customers	各州销售排名、区域对比
mv_category_sales	年-月 × 英语品类	total_gmv, total_orders, avg_price	品类结构、Top 品类
mv_delivery_perf	年-月 × 客户州	avg_delivery_days, on_time_rate, delayed_orders	配送延迟、准时率
mv_seller_perf	年-月 × 卖家	total_gmv, total_orders, avg_review_score	卖家绩效、高差评卖家
mv_payment_dist	年-月 × 支付类型	total_transactions, avg_installments, total_value	支付偏好、平均分期

GMV 计算：各视图均基于 `order_items` 的 `price + freight_value` 汇总，与 `data_dictionary.yaml` 中 `gmv_definition` 一致。

月份键字段：统一为 ``year_month``（格式 `%Y-%m`）。因 MySQL 保留词 `YEAR_MONTH` 存在词法冲突，建表与查询时需使用反引号转义。

4.3 构建与刷新

- 构建脚本：`utils/pre_aggregation.sql`（DROP → CREATE → INSERT）

- 自动化: `utils/db_init.py` 在基表导入完成后自动执行该脚本
- 刷新策略: 数据更新时可 `TRUNCATE` 各 `mv_*` 后重新执行 `INSERT` 部分, 或整库重跑 `db_init.py`

4.4 Agent 查询策略

1. 用户问题维度能匹配上表粒度时, 优先 `SELECT ... FROM mv_*` ;
2. 无法覆盖 (如查某一具体 `order_id` 详情) 时, 回退基表 `JOIN`;
3. Prompt / 路由配置中注入 `data_dictionary.yaml` 的 `materialized_aggregates` 段。

5. 性能优化对比实验

5.1 实验目的

验证预聚合层对高频分析查询的加速效果, 为 Agentic BI 系统中「数据分析 Agent 优先查 `mv_*`」提供量化依据。

5.2 实验环境

- 数据库: MySQL 8.0, 库名 `olist_agentic_bi`
- 客户端: MySQL Workbench
- 对比脚本: `utils/perf_comparison_queries.sql`
- 方法: 同一业务口径下, 分别执行「读预聚合表」与「基表多表 `JOIN` 现算», 记录耗时与 `EXPLAIN` 计划

5.3 实验一: 2017 年各月 GMV 趋势

快查询 (预聚合):

```
1  SELECT `year_month`, SUM(total_gmv) AS gmv_by_month
2  FROM mv_monthly_sales
3  WHERE `year_month` LIKE '2017-%'
4  GROUP BY `year_month`
5  ORDER BY `year_month`;
```

慢查询（基表现算）：

```
1 SELECT DATE_FORMAT(o.order_purchase_timestamp, '%Y-%m') AS ym,
2         SUM(oi.price + oi.freight_value) AS gmv_live
3 FROM orders o
4 JOIN order_items oi ON o.order_id = oi.order_id
5 WHERE o.order_purchase_timestamp >= '2017-01-01'
6       AND o.order_purchase_timestamp < '2018-01-01'
7 GROUP BY ym
8 ORDER BY ym;
```

结果摘要：

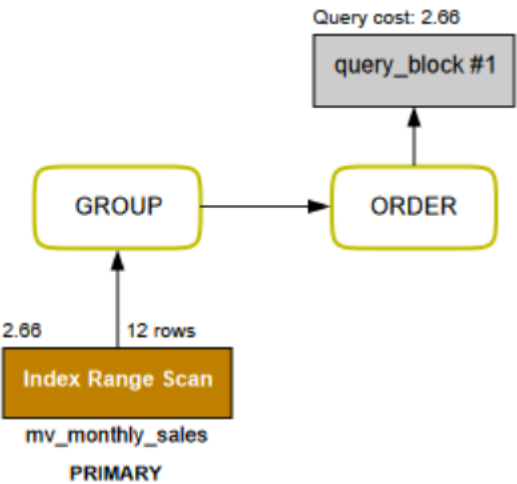
方式	返回行数	客户端耗时 (约)	EXPLAIN 估计扫描 行数	优化器 cost (约)
mv_monthly_sales	12	~0.000 s	12	2.66
orders JOIN order_items	12	~0.875 s	~54,850	82,184

两种查询返回的 2017 年 12 个月 GMV 数值一致（业务口径等价）。快查询仅对预聚合小表做 **Index Range Scan**；慢查询需 **Nested Loop Join** 扫描约 4.7 万订单行并与明细表关联，并产生临时表与 filesort。

例 1 – 快查询结果与耗时：

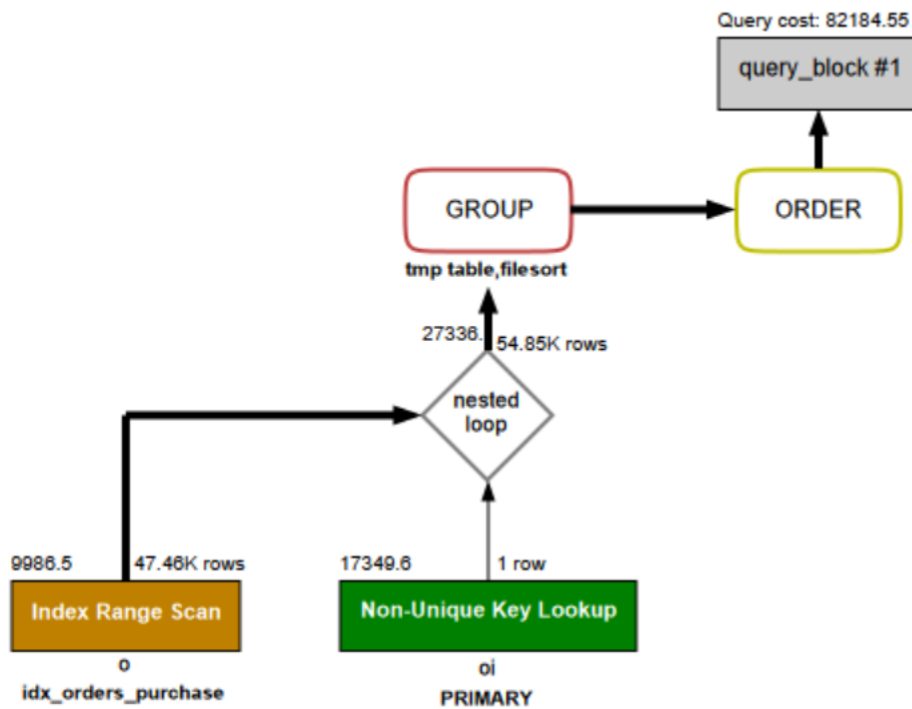


例 1 – 快查询EXPLAIN：



例 1 – 慢查询结果与耗时：

例 1 – 慢查询EXPLAIN:



5.4 实验二：2017 年 Q4 支付方式分布

快查询: `mv_payment_dist`，筛选 ``year_month` BETWEEN '2017-10' AND '2017-12'`，按 `payment_type` 汇总交易笔数。

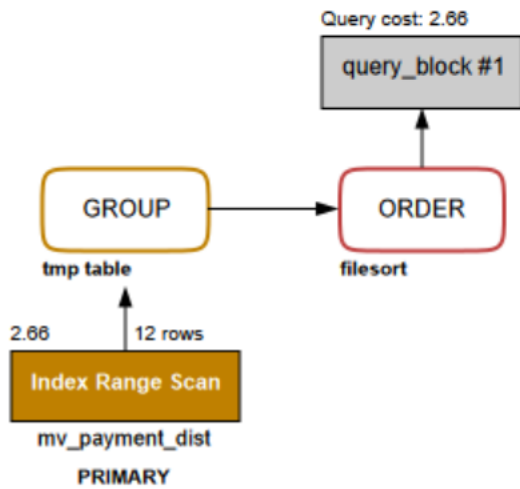
慢查询: `payments JOIN orders`，按下单时间过滤 2017-Q4 后 `GROUP BY payment_type`。

结果摘要:

方式	EXPLAIN 估计扫描行数	优化器 cost (约)
<code>mv_payment_dist</code>	12	2.66
<code>orders JOIN payments</code>	~39,000	20,244

例 2 – 快查询结果与耗时:

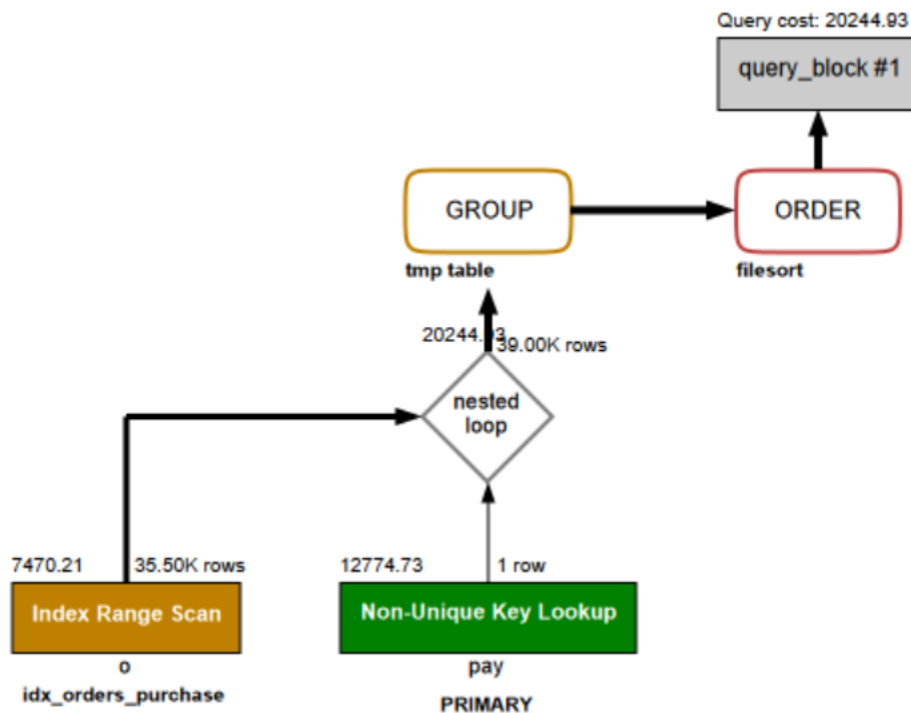
例 2 – 快查询EXPLAIN:



例 2 – 慢查询结果与耗时：

16 16:11:23 SELECT pay.payment_type, COUNT(*) AS tx_cnt FROM payments pay JOIN orders o ON o.ord... 4 row(s) returned 0.141 sec / 0.000 sec

例 2 – 慢查询EXPLAIN：



5.5 结论

1. 在相同分析语义下，预聚合表将 多表 JOIN + 大范围扫描 + 临时聚合 前移到离线批处理，在线查询仅读取行数极少（约十余行）的 `mv_*` 表，耗时与优化器 cost 均显著下降。
2. 例 1 中慢查询耗时约为快路径的 三个数量级以上（0.875 s vs 毫秒级），符合作业对 Pre-Aggregation 加速层的预期。
3. 对本项目 Agent 而言：月度销售、州别、品类、配送、卖家、支付等高频维度应通过 `query_rou`

`ter` 路由至对应 `mv_*`；仅当问题超出预计算粒度时才回退基表，以兼顾响应速度与查询灵活性。